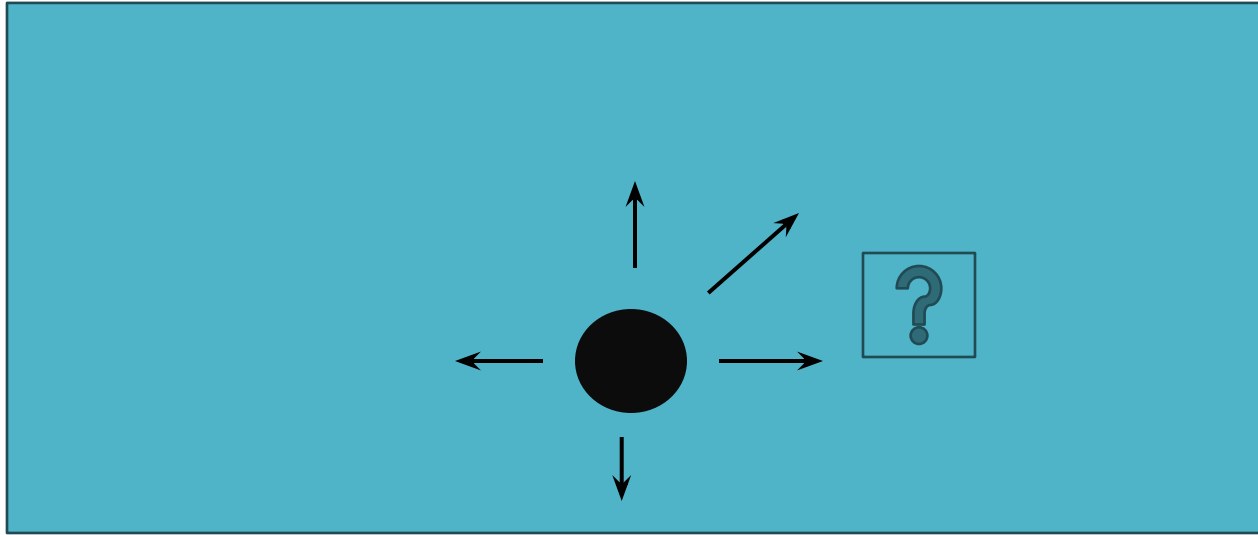
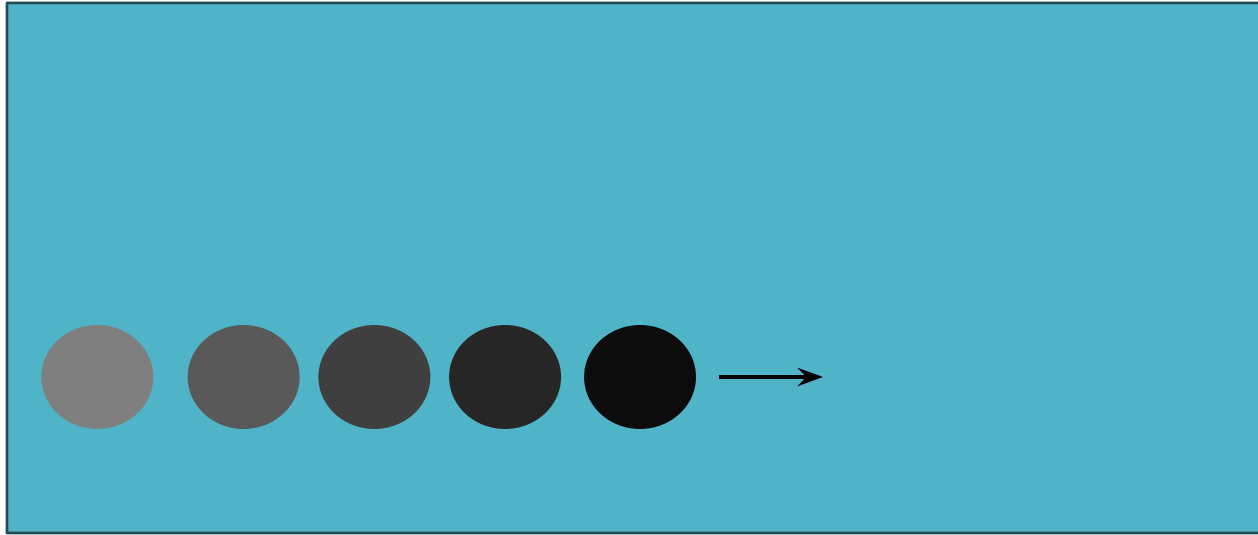


Sequence Modelling

Given the image of the ball, can you predict its next position?



Now?

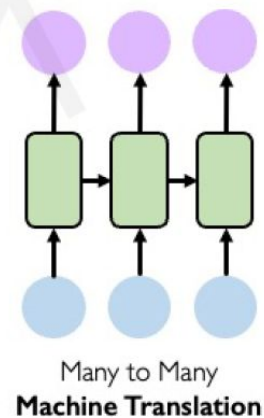
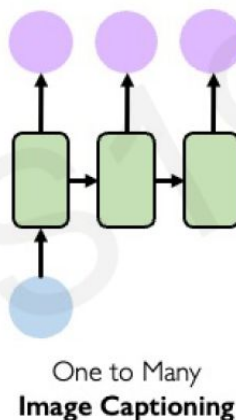
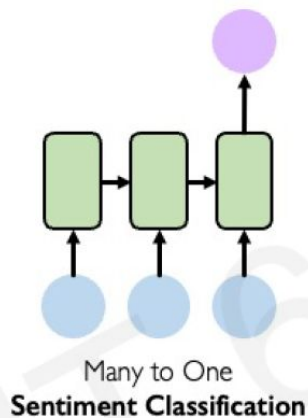
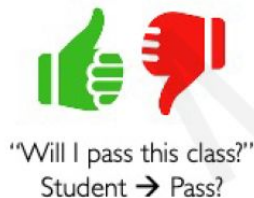
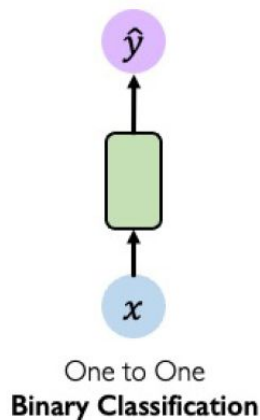


Real world sequences...

- Text
- Speech (audio) data.
- Video frames (video data).
- DNA sequences.
- Stock price data.



Sequence modelling applications



Can a normal neural network understand a sentence if I shuffle the words?

Short answer: NO Because FFNN has no concept of input order.

Why feedforward network will fail?

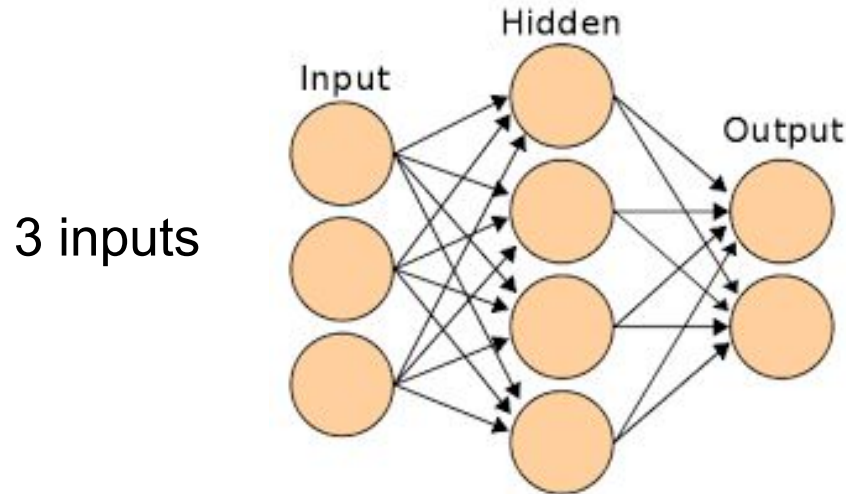
Consider the following set of textual data:

1. "My name is xyz". (Words = 4)
2. "I love India." (Words = 3)
3. "I am playing in the field." (Words = 6)

I want a model to learn and predict from this.

Varying input length

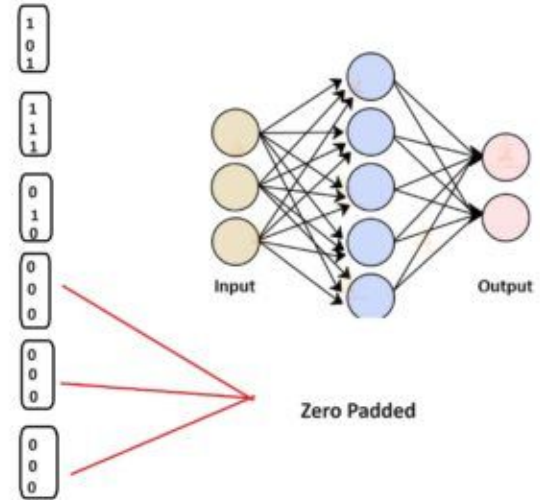
- Input sentences will practically have vastly different lengths.
- So weight matrix will change with the input text size.
- That's impractical for designing.



Unnecessary computation due to zero-pad

- To resolve the previous issue, we can use zero-pad.
- But we'll have to make input size equal to maximum possible input length.
- Count number of words in the longest sentence.

What if the avg length = 50,
but max length = 1000?



What if input length is different for prediction?

- Imagine we have trained our network for maximum 100 input length.
- But we have to predict for an input of length 200.

Will it work?

Ignoring the sequential order of input

- ANN by design doesn't take into consideration the relative order.
- If you shuffle the words of a sentence it will fail.

What do we want from a Sequence model?

- Handle variable length inputs.
- Capture order and temporal dependency.
- Maintain a form of memory/state.
- Share parameters across time (because we don't want different models for different words).
- Learn short-term and long-term patterns.

Computation over time

Computational Graph

- A graph used to represent mathematical expression.
- Equivalent expression: $(a + b) * (b - c)$

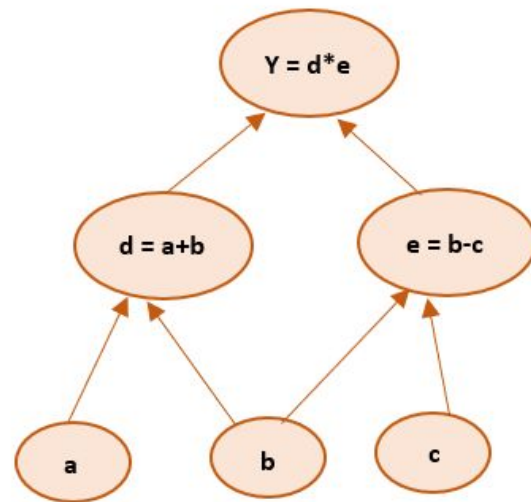
- Consider a dynamical system:

$$s^{(t)} = f(s^{(t-1)}; \theta),$$

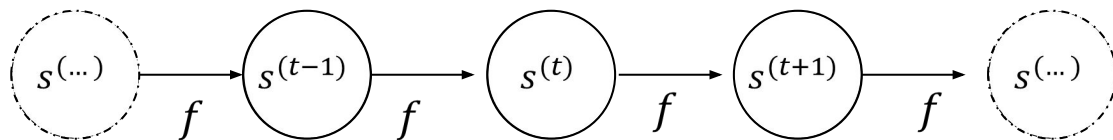
where $s^{(t)}$ = *state of the system at the time t*
and θ is the shared parameter.

- If unfolded to 3 time steps:

$$s^{(3)} = f(s^{(2)}; \theta) = f(f(s^{(1)}; \theta); \theta)$$

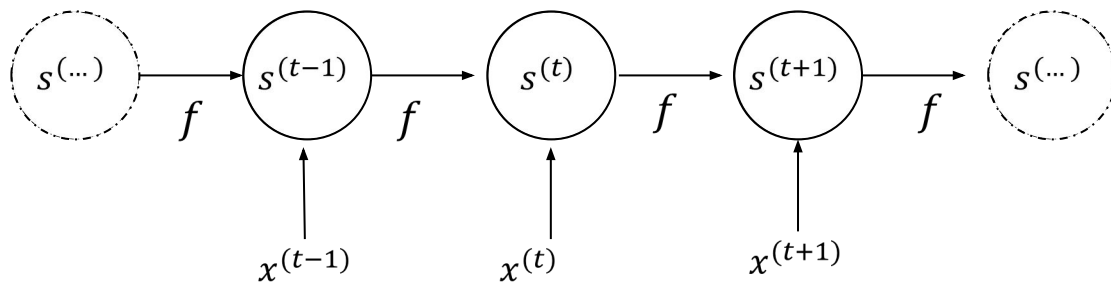


Unfolding

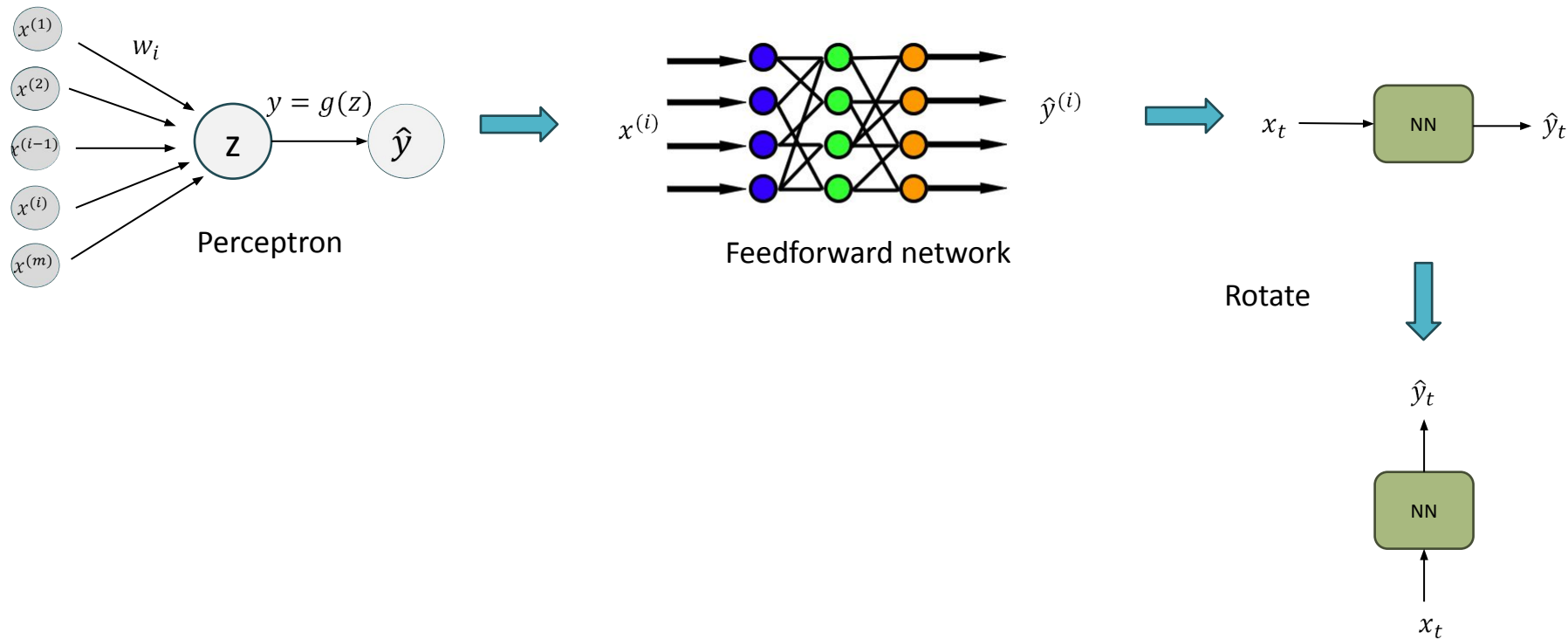


■ But there is no recurrence here.

■ What if?
$$s^{(t)} = f(s^{(t-1)}, x^{(t)}; \theta)$$



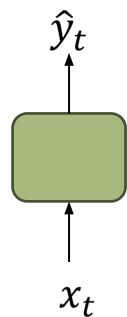
The build up....



Handling multiple time steps

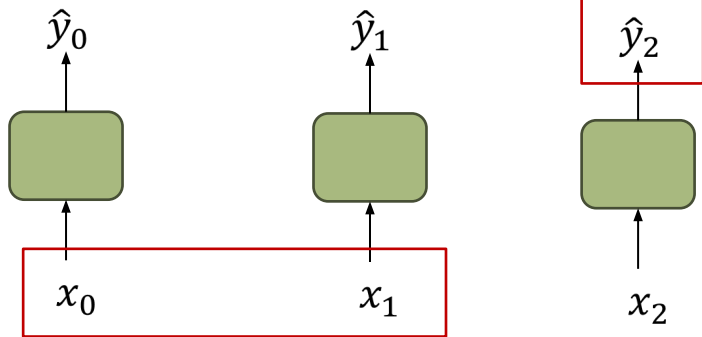
- Can we just repeat the structure to handle multiple time steps?
- NO.
- Because to predict $\hat{y}_2$, it's not considering any previous context.

Output vector



Input vector

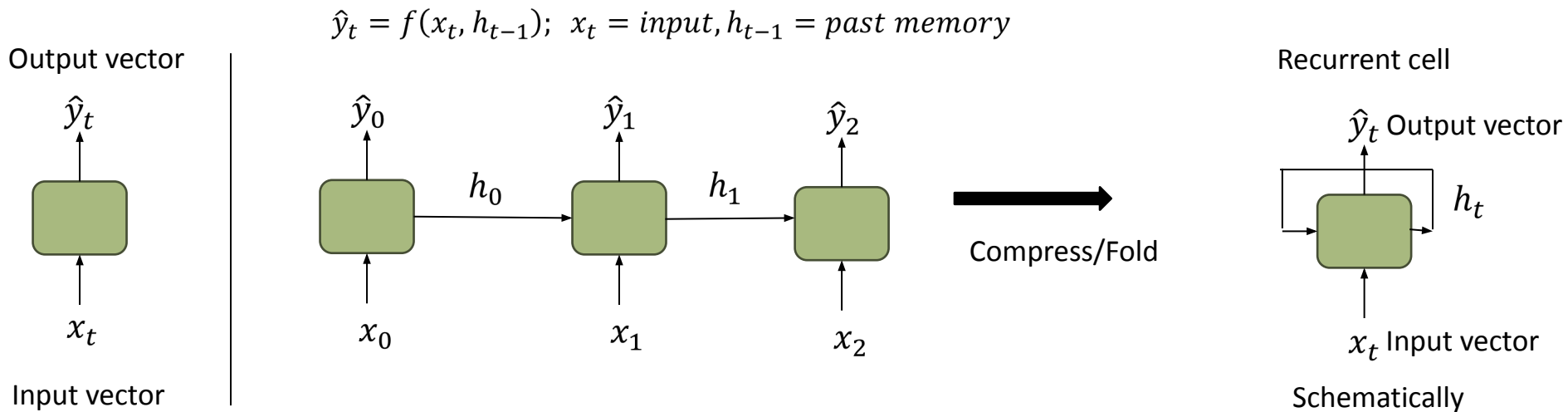
$$\hat{y}_t = f(x_t)$$



Solution?

Neurons with Recurrence

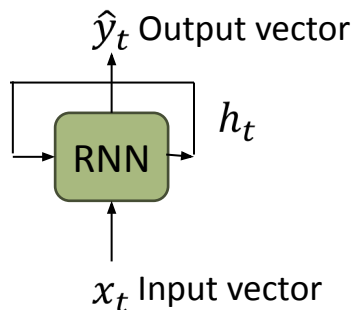
- We need to provide previous context to predict.
- We can call this hidden state (represents previous context/memory).



Recurrent Neural Network (RNN)

- Apply recurrence relation at every time step to process a sequence:
- $\mathbf{h}_t = f_W(\mathbf{x}_t, \mathbf{h}_{t-1})$
- $\mathbf{h}_t$ = cell state (*necessarily lossy summary of past*)
- f_W = function with weights W
- $\mathbf{x}_t$ = input
- $\mathbf{h}_{t-1}$ = old state
- Same function & same set of weights are used at every time step.
- Cell state $\mathbf{h}_t$ is updated at each time step as a sequence is processed.

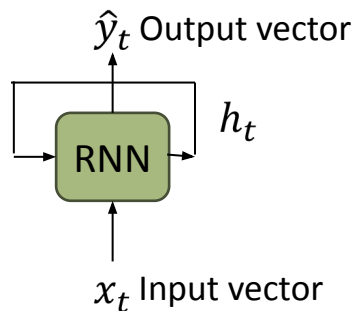
Recurrent cell



The idea of parameter sharing

- If I read a sentence word-by-word, do I change the rules of grammar at every word?
- NO
- RNN works the same way – the rule (weights) stays the same at every time step.
- If weights are not shared:
 - We'll have too many parameters.
 - Cannot generalize to longer sequences..
- Hence Same W is reused at every time step.

RNN



Output vector

$$\hat{y}_t = W_{hy}^T h_t$$

Hidden state

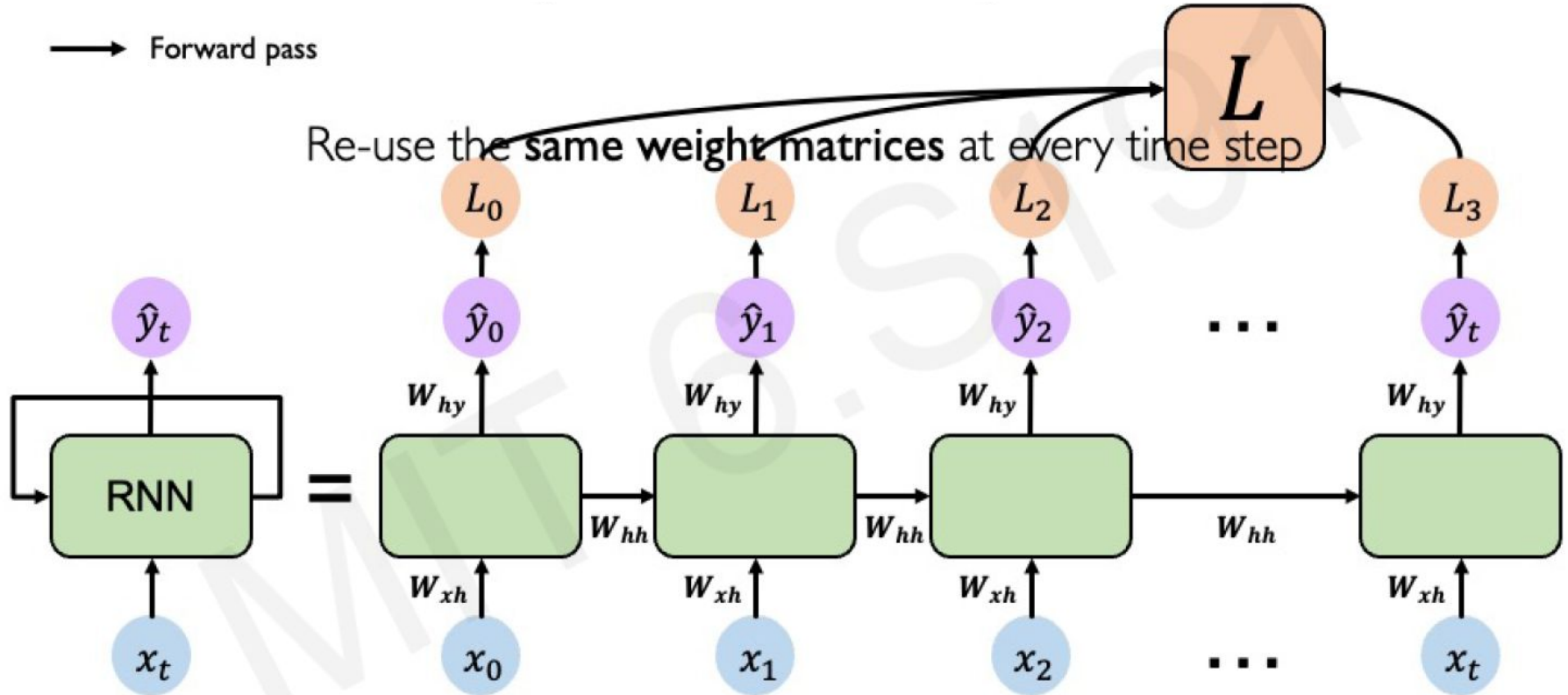
$$h_t = \tanh(W_{hh}^T h_{t-1} + W_{xh}^T x_t)$$

Input vector

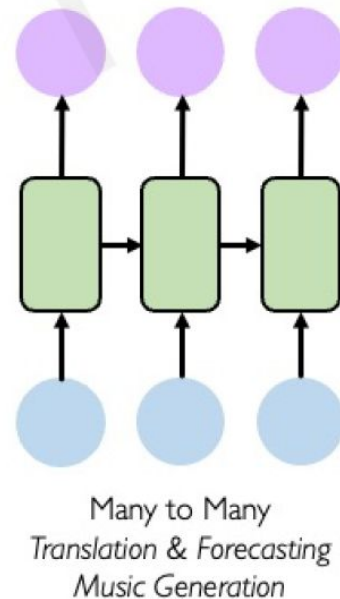
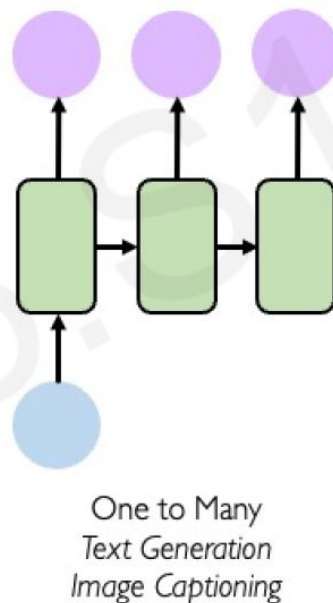
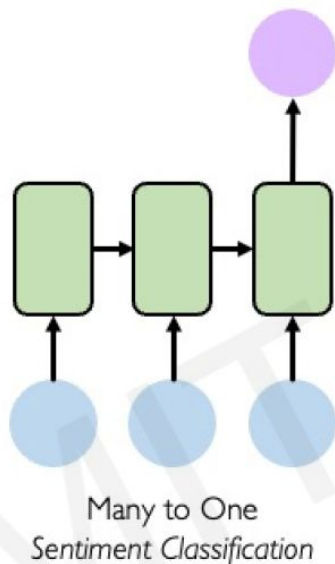
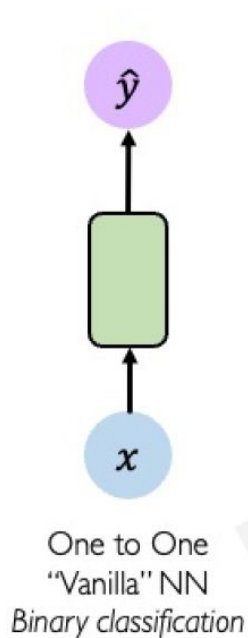
$$x_t$$

RNN: Computational graph across time

→ Forward pass



Types of RNN outputs



Sentiment classification : Example

- Say we have this following dataset of reviews for sentiment classification.

Review	Sentiment
movie was good	1
movie was bad	0
movie was not good	0

t_1 t_2 t_3 t_4

Standard input shape: (timesteps, input_features)

First we have to encode the words.
We'll use one-hot encoding.

Vocabulary size = 5

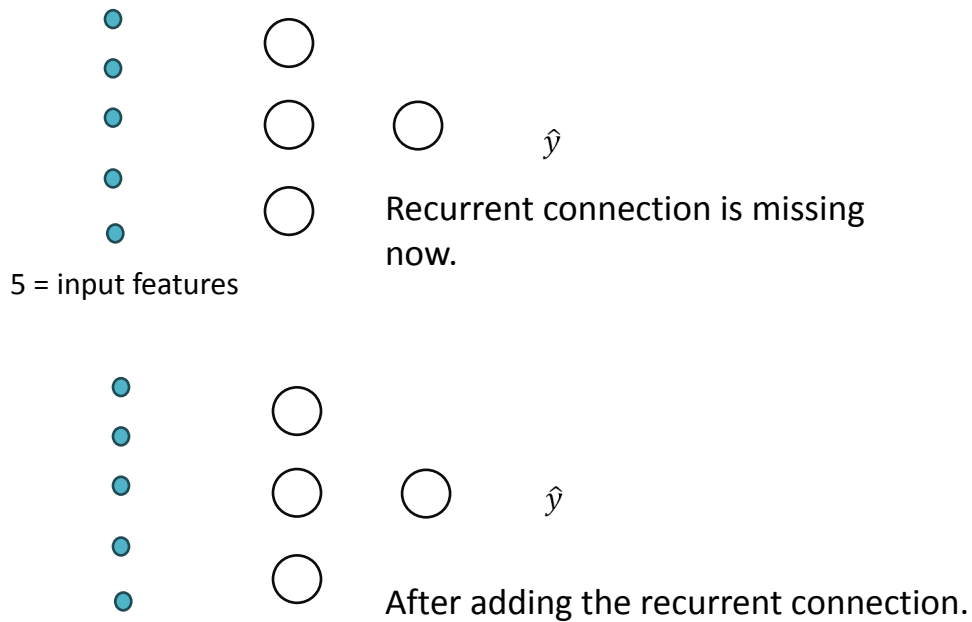
Word	Encoding
movie	[1 0 0 0 0]
was	[0 1 0 0 0]
good	[0 0 1 0 0]
bad	[0 0 0 1 0]
not	[0 0 0 0 1]

First sample: [[1 0 0 0 0], [0 1 0 0 0], [0 0 1 0 0]]

Shape: (3, 5), 3 - time steps, 5 - features

X

	Review	Sentiment
x_1	movie was good	1
x_2	movie was bad	0
x_3	movie was not good	0



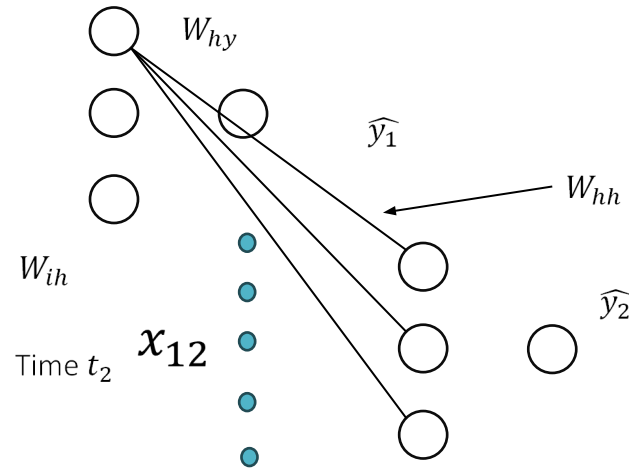
Let's simplify as per
time....

X

	Review	Sentiment
x_1	movie was good	1
x_2	movie was bad	0
x_3	movie was not good	0

5 = input features

x_{11}
Time t_1



And so
on....

Weights:

$$W_{ih} = 5 \times 3 = 15$$

$$W_{hy} = 3 \times 1 = 3$$

$$W_{hh} = 3 \times 3 = 9$$

Biases:

3 bias for hidden layer.

1 bias for output neuron.

Total parameters:

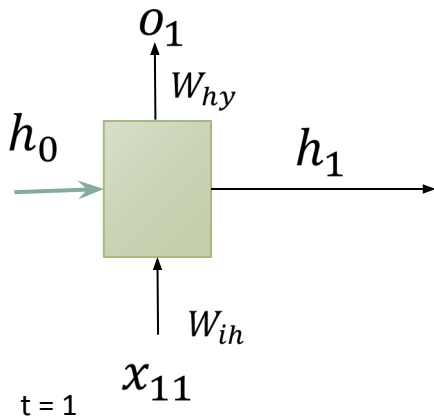
$$15 + 3 + 9 + 4 = 31$$

Let's simplify things for forward pass..

X

	Review				Sentimen
	x_{11}	x_{12}	x_{13}		t
x_1	x_{11}	x_{12}	x_{13}		1
x_2	x_{21}	x_{22}	x_{23}		0
x_3	x_{31}	x_{32}	x_{33}	x_{34}	0

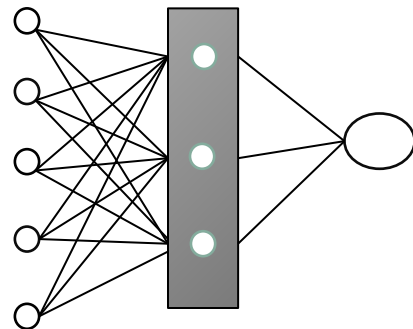
x_{ij} – 5 dimentional vector here



$$h_1 = f_1(W_{hh}h_0 + W_{ih}x_{11} + b_1)$$

$$o_1 = f_2(W_{hy}h_1 + b_2)$$

f_1, f_2 are activation functions.



Let's simplify things for forward pass..

X

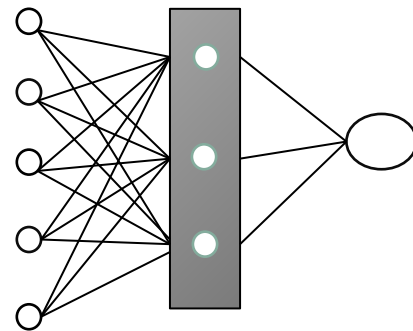
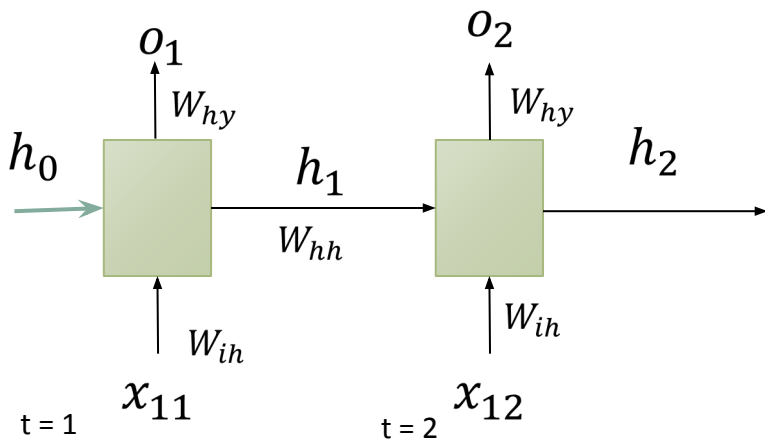
	Review				Sentimen
	x_{11}	x_{12}	x_{13}		t
x_1	x_{11}	x_{12}	x_{13}		1
x_2	x_{21}	x_{22}	x_{23}		0
x_3	x_{31}	x_{32}	x_{33}	x_{34}	0

x_{ij} – 5 dimentional vector here

$$h_2 = f_1(W_{hh}h_1 + W_{ih}x_{12} + b_1)$$

$$o_2 = f_2(W_{hy}h_2 + b_2)$$

f_1, f_2 are activation functions.



Let's simplify things for forward pass..

X

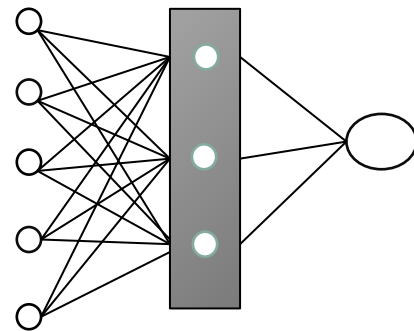
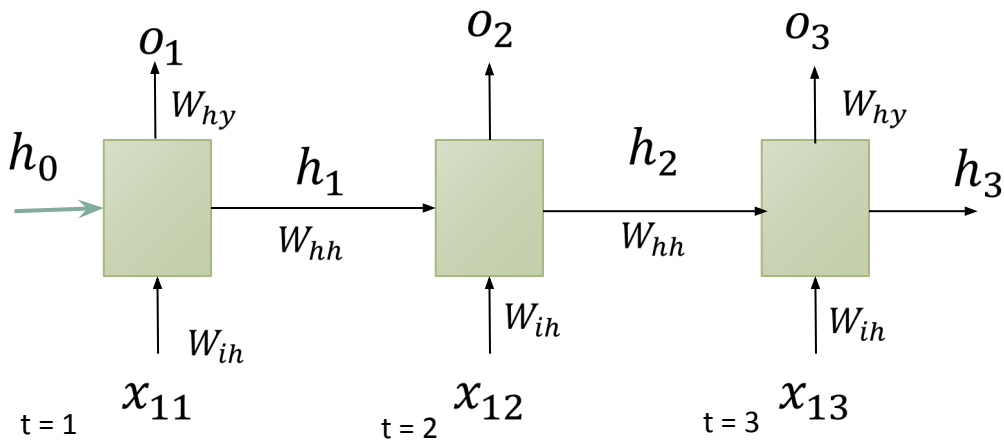
	Review				Sentimen
	x_{11}	x_{12}	x_{13}		t
x_1	x_{11}	x_{12}	x_{13}		1
x_2	x_{21}	x_{22}	x_{23}		0
x_3	x_{31}	x_{32}	x_{33}	x_{34}	0

x_{ij} – 5 dimentional vector here

$$h_3 = f_1(W_{hh}h_2 + W_{ih}x_{13} + b_1)$$

$$o_3 = f_2(W_{hy}h_3 + b_2)$$

f_1, f_2 are activation functions.



Let's simplify things for forward pass..

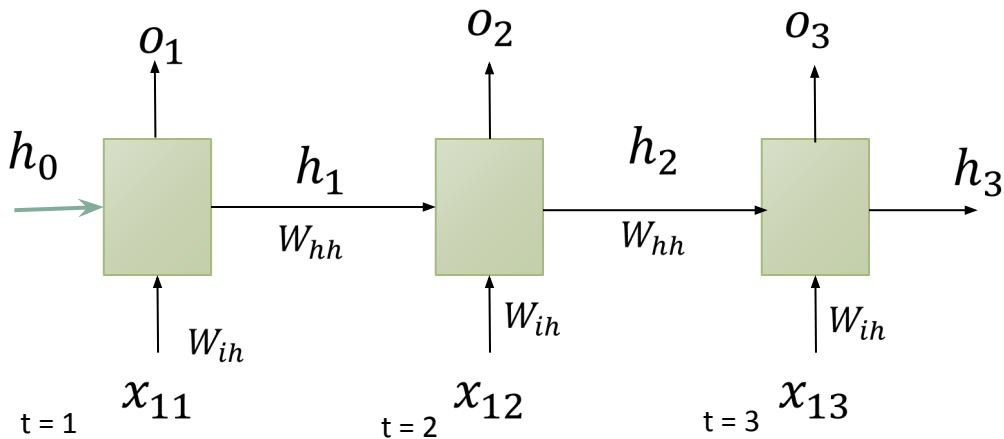
X

	Review				Sentimen
x_1	x_{11}	x_{12}	x_{13}		t
x_2	x_{21}	x_{22}	x_{23}		1
x_3	x_{31}	x_{32}	x_{33}	x_{34}	0
					0

x_{ij} – 5 dimensional vector here

Clearly observation points:

- Parameter sharing.
- Sequential processing of inputs
- Previous context being utilized for computation



Training RNNs: Backpropagation through time

Sentiment classification example

Review	Sentiment
cat mat rat	1
cat mat mat	1
rat mat cat	0

Many to one output

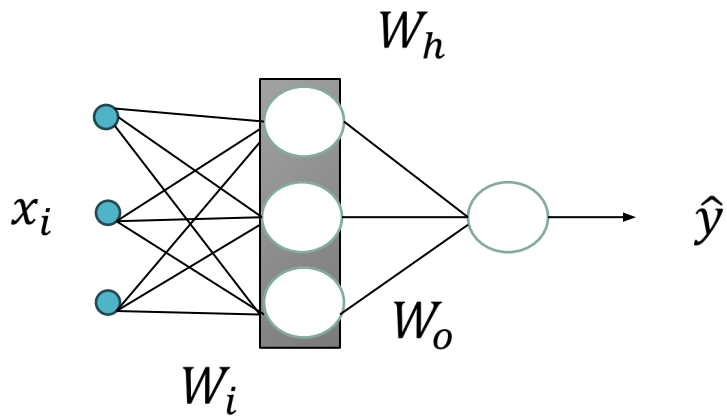
One-hot encoding

cat [1 0 0]

mat [0 1 0]

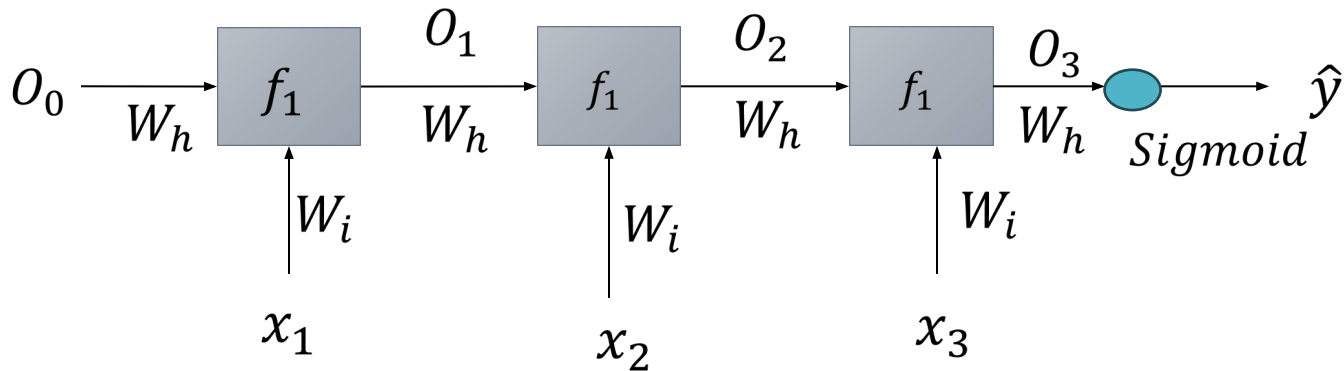
rat [0 0 1]

Review	Sentiment
[1 0 0] [0 1 0] [0 0 1]	1
[1 0 0] [0 1 0] [0 1 0]	1
[0 0 1] [0 1 0] [1 0 0]	0



Let's ignore bias for now (for simplicity)

Unfolded for inputs over time



Actual output: y Prediction: $\hat{y}$ Loss $L = \hat{y} - y$

$$O_1 = f_1(W_i x_1 + W_h O_0)$$

$$O_2 = f_1(W_i x_2 + W_h O_1)$$

$$O_3 = f_1(W_i x_3 + W_h O_2)$$

$\hat{y} = \sigma(W_0 O_3)$, here σ is a sigmoid activation.

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Gradient updates

$$W_i = W_i - \alpha \frac{\partial L}{\partial W_i}$$

$$W_h = W_h - \alpha \frac{\partial L}{\partial W_h}$$

$$W_o = W_o - \alpha \frac{\partial L}{\partial W_o}$$

If we derive $\frac{\partial L}{\partial W_i}$:

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial W_i} + \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial O_2} * \frac{\partial O_2}{\partial W_i} + \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial O_2} * \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial W_i}$$

3-terms 4-terms 5-terms

 ↑ ↑ ↑

 t=3 t=2 t=1

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial W_i} + \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial O_2} * \frac{\partial O_2}{\partial W_i} + \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial O_2} * \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial W_i}$$



This relates to short-term
Dependency.



This relates to long-term
Dependency.

Only 3 time-sequences.

What if we have 100 time-steps in each sequence???

The last term will keep getting

$$\frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_3} * \frac{\partial O_3}{\partial O_2} * \frac{\partial O_2}{\partial O_1} * \frac{\partial O_1}{\partial W_i}$$

Let's focus on the last term.

$$\frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * \frac{\partial \mathbf{O}_{100}}{\partial \mathbf{O}_{99}} * \dots * \frac{\partial \mathbf{O}_3}{\partial \mathbf{O}_2} * \frac{\partial \mathbf{O}_2}{\partial \mathbf{O}_1} * \frac{\partial O_1}{\partial W_i}$$

Expand for 100 timesteps.

$$\frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * \prod_{t=2}^{100} \frac{\partial \mathbf{O}_t}{\partial \mathbf{O}_{t-1}} * \frac{\partial O_1}{\partial W_i}$$

$$\begin{aligned}
 \frac{\partial O_t}{\partial O_{t-1}} &= \frac{\partial}{\partial O_{t-1}} [f(W_i x_t + W_h O_{t-1})] && \text{Apply chain rule} \\
 &= f'(W_i x_t + W_h O_{t-1}) * \frac{\partial}{\partial O_{t-1}} [W_i x_t + W_h O_{t-1}] \\
 &= f'(W_i x_t + W_h O_{t-1}) * W_h \\
 &= f'(z) * W_h \text{ (for brevity)}
 \end{aligned}$$

By default, *tanh* is used as activation for hidden layer in RNN.

$$f'(z) = \frac{\partial}{\partial z} (\tanh z) = 1 - (\tanh z)^2$$

Range of $f'(z)$ is $(0, 1]$.

Let's use the previous result now...

$$\begin{aligned} & \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * \prod_{t=2}^{100} \frac{\partial \mathbf{O}_t}{\partial \mathbf{O}_{t-1}} * \frac{\partial O_1}{\partial W_i} \\ &= \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * \prod_{t=2}^{100} (f'(z) * W_h) * \frac{\partial O_1}{\partial W_i} \\ &= \frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * [(f'(z) * W_h)]^{99} * \frac{\partial O_1}{\partial W_i} \end{aligned}$$

Exponentiation is a very unstable behavior.

$$0.99^{200} \approx 0.134$$

$$1^{200} = 1$$

$$1.01^{200} \approx 13$$

Let's use the previous result now...

$$\frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * [(f'(z) * W_h)]^{99} * \frac{\partial O_1}{\partial W_i}$$

- If λ_i be the eigenvalue of W_h , then W_h will either vanish or explode:
 - If $|\lambda_i| < 1$
 - If $|\lambda_i| > 1$
- After eigen-decomposition of W , $W^k = V \text{diag}(\lambda)^k V^{-1}$

Let's use the previous result now...

$$\frac{\partial L}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial O_{100}} * [(f'(z) * W_h)]^{99} * \frac{\partial O_1}{\partial W_i}$$

- If λ_i be the eigenvalue of W_h , then W_h will either vanish or explode:

- If $|\lambda_i| < 1$

Vanishing Gradient

- If $|\lambda_i| > 1$

Exploding Gradient

- After eigen-decomposition of W , $W^k = V \text{diag}(\lambda)^k V^{-1}$

Many-to-many scenario

